

Software Requirements Specification for Projectile

Samuel J. Crawford, Brooks MacLachlan, and W. Spencer Smith

November 3, 2025

Contents

1	Reference Material	3
1.1	Table of Units	3
1.2	Table of Symbols	3
1.3	Abbreviations and Acronyms	4
2	Introduction	5
2.1	Purpose of Document	5
2.2	Scope of Requirements	6
2.3	Characteristics of Intended Reader	6
2.4	Organization of Document	6
3	General System Description	6
3.1	System Context	6
3.2	User Characteristics	7
3.3	System Constraints	7
4	Specific System Description	7
4.1	Problem Description	8
4.1.1	Terminology and Definitions	8
4.1.2	Physical System Description	8
4.1.3	Goal Statements	9
4.2	Solution Characteristics Specification	9
4.2.1	Assumptions	9
4.2.2	Theoretical Models	10
4.2.3	General Definitions	11
4.2.4	Data Definitions	15
4.2.5	Instance Models	17
4.2.6	Data Constraints	22
4.2.7	Properties of a Correct Solution	23

5	Requirements	23
5.1	Functional Requirements	23
5.2	Non-Functional Requirements	24
6	Traceability Matrices and Graphs	24
7	Values of Auxiliary Constants	30
8	References	30

1 Reference Material

This section records information for easy reference.

1.1 Table of Units

The unit system used throughout is SI (Système International d'Unités). In addition to the basic units, several derived units are also used. For each unit, the [Table of Units](#) lists the symbol, a description, and the SI name.

Table 1: Table of Units

Symbol	Description	SI Name
m	length	metre
rad	angle	radian
s	time	second

1.2 Table of Symbols

The symbols used in this document are summarized in the [Table of Symbols](#) along with their units. Throughout the document, symbols in bold will represent vectors, and scalars otherwise. The symbols are listed in alphabetical order. For vector quantities, the units shown are for each component of the vector.

Table 2: Table of Symbols

Symbol	Description	Units
a	Scalar acceleration	$\frac{\text{m}}{\text{s}^2}$
a^c	Constant acceleration	$\frac{\text{m}}{\text{s}^2}$
a_x	x -component of acceleration	$\frac{\text{m}}{\text{s}^2}$
a_x^c	x -component of constant acceleration	$\frac{\text{m}}{\text{s}^2}$
a_y	y -component of acceleration	$\frac{\text{m}}{\text{s}^2}$
a_y^c	y -component of constant acceleration	$\frac{\text{m}}{\text{s}^2}$
$\mathbf{a}(t)$	Acceleration	$\frac{\text{m}}{\text{s}^2}$
$\mathbf{a}^c$	Constant acceleration vector	$\frac{\text{m}}{\text{s}^2}$
d_{offset}	Distance between the target position and the landing position	m
g	Magnitude of gravitational acceleration	$\frac{\text{m}}{\text{s}^2}$
p	Scalar position	m

Continued on next page

Table 2: Table of Symbols (Continued)

Symbol	Description	Units
$p(t)$	1D position	m
p^i	Initial position	m
p_{land}	Landing position	m
p_{target}	Target position	m
p_x	x -component of position	m
p_x^i	x -component of initial position	m
p_y	y -component of position	m
p_y^i	y -component of initial position	m
$\mathbf{p}(t)$	Position	m
t	Time	s
t_{flight}	Flight duration	s
v	Speed	$\frac{\text{m}}{\text{s}}$
$v(t)$	1D speed	$\frac{\text{m}}{\text{s}}$
v^i	Initial speed	$\frac{\text{m}}{\text{s}}$
v_{launch}	Launch speed	$\frac{\text{m}}{\text{s}}$
v_x	x -component of velocity	$\frac{\text{m}}{\text{s}}$
v_x^i	x -component of initial velocity	$\frac{\text{m}}{\text{s}}$
v_y	y -component of velocity	$\frac{\text{m}}{\text{s}}$
v_y^i	y -component of initial velocity	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}(t)$	Velocity	$\frac{\text{m}}{\text{s}}$
$\mathbf{v}^i$	Initial velocity	$\frac{\text{m}}{\text{s}}$
ε	Hit tolerance	—
θ	Launch angle	rad
π	Ratio of circumference to diameter for any circle	—

1.3 Abbreviations and Acronyms

Table 3: Abbreviations and Acronyms

Abbreviation	Full Form
1D	One-Dimensional
2D	Two-Dimensional
A	Assumption
DD	Data Definition
GD	General Definition
GS	Goal Statement
IM	Instance Model
PS	Physical System Description
R	Requirement
RefBy	Referenced by
Refname	Reference Name
SRS	Software Requirements Specification
TM	Theoretical Model
Uncert.	Typical Uncertainty

2 Introduction

Projectile motion is a common problem in physics. Therefore, it is useful to have a program to solve and model these types of problems. Common examples of projectile motion include ballistics problems (missiles, bullets, etc.) and the flight of balls in various sports (baseball, golf, football, etc.). The document describes the program called Projectile , which is based on the original, manually created version of [Projectile](#).

The following section provides an overview of the Software Requirements Specification (SRS) for Projectile. This section explains the purpose of this document, the scope of the requirements, the characteristics of the intended reader, and the organization of the document.

2.1 Purpose of Document

The primary purpose of this document is to record the requirements of Projectile. Goals, assumptions, theoretical models, definitions, and other model derivation information are specified, allowing the reader to fully understand and verify the purpose and scientific basis of Projectile. With the exception of **system constraints**, this SRS will remain abstract, describing what problem is being solved, but not how to solve it.

This document will be used as a starting point for subsequent development phases, including writing the design specification and the software verification and validation plan.

The design document will show how the requirements are to be realized, including decisions on the numerical algorithms and programming environment. The verification and validation plan will show the steps that will be used to increase confidence in the software documentation and the implementation. Although the SRS fits in a series of documents that follow the so-called waterfall model, the actual development process is not constrained in any way. Even when the waterfall model is not followed, as Parnas and Clements point out [6], the most logical way to present the documentation is still to “fake” a rational design process.

2.2 Scope of Requirements

The scope of the requirements includes the analysis of a two-dimensional (2D) projectile motion problem with constant acceleration.

2.3 Characteristics of Intended Reader

Reviewers of this documentation should have an understanding of undergraduate level 1 physics and undergraduate level 1 calculus. The users of Projectile can have a lower level of expertise, as explained in [Sec:User Characteristics](#).

2.4 Organization of Document

The organization of this document follows the template for an SRS for scientific computing software proposed by [5], [8], [9], and [7]. The presentation follows the standard pattern of presenting goals, theories, definitions, and assumptions. For readers that would like a more bottom up approach, they can start reading the [instance models](#) and trace back to find any additional information they require.

The [goal statements](#) are refined to the theoretical models and the [theoretical models](#) to the [instance models](#).

3 General System Description

This section provides general information about the system. It identifies the interfaces between the system and its environment, describes the user characteristics, and lists the system constraints.

3.1 System Context

[Fig:sysCtxDiag](#) shows the system context. A circle represents an entity external to the software, the user in this case. A rectangle represents the software system itself (Projectile). Arrows are used to show the data flow between the system and its environment.



Figure 1: System Context

The interaction between the product and the user is through an application programming interface. The responsibilities of the user and the system are as follows:

- User Responsibilities
 - Provide initial conditions of the physical state of the motion and the input data related to the Projectile, ensuring no errors in the data entry.
 - Ensure that consistent units are used for input variables.
 - Ensure required **software assumptions** are appropriate for any particular problem input to the software.
- Projectile Responsibilities
 - Detect data type mismatch, such as a string of characters input instead of a floating point number.
 - Determine if the inputs satisfy the required physical and software constraints.
 - Calculate the required outputs.

3.2 User Characteristics

The end user of Projectile should have an understanding of high school physics and high school calculus.

3.3 System Constraints

There are no system constraints.

4 Specific System Description

This section first presents the problem description, which gives a high-level view of the problem to be solved. This is followed by the solution characteristics specification, which presents the assumptions, theories, and definitions that are used.

4.1 Problem Description

A system is needed to predict whether a launched projectile hits its target.

4.1.1 Terminology and Definitions

This subsection provides a list of terms that are used in the subsequent sections and their meaning, with the purpose of reducing ambiguity and making it easier to correctly understand the requirements.

- Launcher: Where the projectile is launched from and the device that does the launching.
- Projectile: The object to be launched at the target.
- Target: Where the projectile should be launched to.
- Gravity: The force that attracts one physical body with mass to another.
- Cartesian coordinate system: A coordinate system that specifies each point uniquely in a plane by a set of numerical coordinates, which are the signed distances to the point from two fixed perpendicular oriented lines, measured in the same unit of length (from [2]).
- Rectilinear: Occurring in one dimension.

4.1.2 Physical System Description

The physical system of Projectile, as shown in **Fig:Launch**, includes the following elements:

PS1: The launcher.

PS2: The projectile (with initial velocity $\mathbf{v}^i$ and launch angle θ).

PS3: The target.

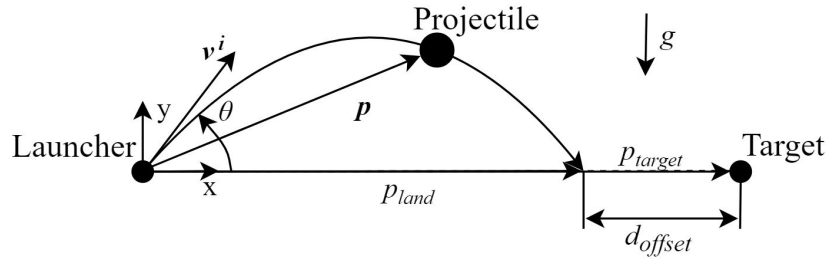


Figure 2: The physical system

4.1.3 Goal Statements

Given the initial velocity vector of the projectile and the geometric layout of the launcher and target, the goal statement is:

targetHit: Determine if the projectile hits the target.

4.2 Solution Characteristics Specification

The instance models that govern Projectile are presented in the [Instance Model Section](#). The information to understand the meaning of the instance models and their derivation is also presented, so that the instance models can be verified.

4.2.1 Assumptions

This section simplifies the original problem and helps in developing the theoretical models by filling in the missing information for the physical system. The assumptions refine the scope by providing more detail.

twoDMotion: The projectile motion is two-dimensional (2D). (RefBy: [GD:velVec](#) and [GD:posVec](#).)

cartSyst: A Cartesian coordinate system is used (from [A:neglectCurv](#)). (RefBy: [GD:velVec](#) and [GD:posVec](#).)

yAxisGravity: The direction of the y -axis is directed opposite to gravity. (RefBy: [IM:calOfLandingDist](#), [IM:calOfLandingTime](#), and [A:accelYGravity](#).)

launchOrigin: The launcher is coincident with the origin. (RefBy: [IM:calOfLandingDist](#) and [IM:calOfLandingTime](#).)

targetXAxis: The target lies on the x -axis (from [A:neglectCurv](#)). (RefBy: [IM:calOfLandingTime](#).)

posXDirection: The positive x -direction is from the launcher to the target. (RefBy: [IM:offsetIM](#), [IM:calOfLandingDist](#), and [IM:calOfLandingTime](#).)

constAccel: The acceleration is constant (from [A:accelXZero](#), [A:accelYGravity](#), [A:neglectDrag](#), and [A:freeFlight](#)). (RefBy: [GD:velVec](#) and [GD:posVec](#).)

accelXZero: The acceleration in the x -direction is zero. (RefBy: [IM:calOfLandingDist](#) and [A:constAccel](#).)

accelYGravity: The acceleration in the y -direction is the acceleration due to gravity (from [A:yAxisGravity](#)). (RefBy: [IM:calOfLandingTime](#) and [A:constAccel](#).)

neglectDrag: Air drag is neglected. (RefBy: [A:constAccel](#).)

pointMass: The size and shape of the projectile are negligible, so that it can be modelled as a point mass. (RefBy: [GD:rectVel](#) and [GD:rectPos](#).)

freeFlight: The flight is free; there are no collisions during the trajectory of the projectile. (RefBy: [A:constAccel](#).)

neglectCurv: The distance is small enough that the curvature of the celestial body can be neglected. (RefBy: [A:targetXAxis](#) and [A:cartSyst](#).)

timeStartZero: Time starts at zero. (RefBy: [GD:velVec](#), [GD:rectVel](#), [GD:rectPos](#), [GD:posVec](#), and [IM:calOfLandingTime](#).)

gravAccelValue: The acceleration due to gravity is assumed to have the value provided in the section for [Values of Auxiliary Constants](#). (RefBy: [IM:calOfLandingDist](#) and [IM:calOfLandingTime](#).)

4.2.2 Theoretical Models

This section focuses on the general equations and laws that Projectile is based on.

Refname	TM:acceleration
Label	Acceleration
Equation	$\mathbf{a}(t) = \frac{d\mathbf{v}(t)}{dt}$
Description	$\mathbf{a}(t)$ is the acceleration ($\frac{\text{m}}{\text{s}^2}$) t is the time (s) $\mathbf{v}(t)$ is the velocity ($\frac{\text{m}}{\text{s}}$)
Source	[1]
RefBy	GD:rectVel

Refname	TM:velocity
Label	Velocity
Equation	$\mathbf{v}(t) = \frac{d\mathbf{p}(t)}{dt}$
Description	$\mathbf{v}(t)$ is the velocity ($\frac{\text{m}}{\text{s}}$) t is the time (s) $\mathbf{p}(t)$ is the position (m)
Source	[3]
RefBy	GD:rectPos

4.2.3 General Definitions

This section collects the laws and equations that will be used to build the instance models.

Refname	GD:rectVel
Label	Rectilinear velocity as a function of time for constant acceleration
Units	$\frac{\text{m}}{\text{s}}$
Equation	$v(t) = v^i + a^c t$
Description	$v(t)$ is the 1D speed ($\frac{\text{m}}{\text{s}}$) v^i is the initial speed ($\frac{\text{m}}{\text{s}}$) a^c is the constant acceleration ($\frac{\text{m}}{\text{s}^2}$) t is the time (s)
Source	[4, (pg. 8)]
RefBy	GD:velVec and GD:rectPos

Detailed derivation of rectilinear velocity: Assume we have rectilinear motion of a particle (of negligible size and shape, from [A:pointMass](#)); that is, motion in a straight line. The velocity is v and the acceleration is a . The motion in [TM:acceleration](#) is now one-dimensional with a constant acceleration, represented by a^c . The initial velocity (at $t = 0$, from [A:timeStartZero](#)) is represented by v^i . From [TM:acceleration](#) in 1D, and using the above symbols we have:

$$a^c = \frac{dv}{dt}$$

Rearranging and integrating, we have:

$$\int_{v^i}^v 1 dv = \int_0^t a^c dt$$

Performing the integration, we have the required equation:

$$v(t) = v^i + a^c t$$

Refname	GD:rectPos
Label	Rectilinear position as a function of time for constant acceleration
Units	m
Equation	$p(t) = p^i + v^i t + \frac{a^c t^2}{2}$
Description	$p(t)$ is the 1D position (m) p^i is the initial position (m) v^i is the initial speed ($\frac{\text{m}}{\text{s}}$) t is the time (s) a^c is the constant acceleration ($\frac{\text{m}}{\text{s}^2}$)
Source	[4, (pg. 8)]
RefBy	GD:posVec

Detailed derivation of rectilinear position: Assume we have rectilinear motion of a particle (of negligible size and shape, from [A:pointMass](#)); that is, motion in a straight line. The position is p and the velocity is v . The motion in [TM:velocity](#) is now one-dimensional.

The initial position (at $t = 0$, from **A:timeStartZero**) is represented by p^i . From **TM:velocity** in 1D, and using the above symbols we have:

$$v = \frac{dp}{dt}$$

Rearranging and integrating, we have:

$$\int_{p^i}^p 1 dp = \int_0^t v dt$$

From **GD:rectVel**, we can replace v :

$$\int_{p^i}^p 1 dp = \int_0^t v^i + a^c t dt$$

Performing the integration, we have the required equation:

$$p(t) = p^i + v^i t + \frac{a^c t^2}{2}$$

Refname	GD:velVec
Label	Velocity vector as a function of time for 2D motion under constant acceleration
Units	$\frac{m}{s}$
Equation	$\mathbf{v}(t) = \begin{bmatrix} v_x^i + a_x^c t \\ v_y^i + a_y^c t \end{bmatrix}$
Description	$\mathbf{v}(t)$ is the velocity ($\frac{m}{s}$) v_x^i is the x-component of initial velocity ($\frac{m}{s}$) a_x^c is the x-component of constant acceleration ($\frac{m}{s^2}$) t is the time (s) v_y^i is the y-component of initial velocity ($\frac{m}{s}$) a_y^c is the y-component of constant acceleration ($\frac{m}{s^2}$)
Source	—
RefBy	

Detailed derivation of velocity vector: For a two-dimensional Cartesian coordinate system (**A:twoDMotion** and **A:cartSyst**), we can represent the velocity vector as $\mathbf{v}(t) = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$

and the acceleration vector as $\mathbf{a}(t) = \begin{bmatrix} a_x \\ a_y \end{bmatrix}$. The acceleration is assumed to be constant (**A:constAccel**) and the constant acceleration vector is represented as $\mathbf{a}^c = \begin{bmatrix} a_x^c \\ a_y^c \end{bmatrix}$. The initial velocity (at $t = 0$, from **A:timeStartZero**) is represented by $\mathbf{v}^i = \begin{bmatrix} v_x^i \\ v_y^i \end{bmatrix}$. Since we have a Cartesian coordinate system, **GD:rectVel** can be applied to each coordinate of the velocity vector to yield the required equation:

$$\mathbf{v}(t) = \begin{bmatrix} v_x^i + a_x^c t \\ v_y^i + a_y^c t \end{bmatrix}$$

Refname	GD:posVec
Label	Position vector as a function of time for two-dimensional motion under constant acceleration
Units	m
Equation	$\mathbf{p}(t) = \begin{bmatrix} p_x^i + v_x^i t + \frac{a_x^c t^2}{2} \\ p_y^i + v_y^i t + \frac{a_y^c t^2}{2} \end{bmatrix}$
Description	$\mathbf{p}(t)$ is the position (m) p_x^i is the x-component of initial position (m) v_x^i is the x-component of initial velocity ($\frac{\text{m}}{\text{s}}$) t is the time (s) a_x^c is the x-component of constant acceleration ($\frac{\text{m}}{\text{s}^2}$) p_y^i is the y-component of initial position (m) v_y^i is the y-component of initial velocity ($\frac{\text{m}}{\text{s}}$) a_y^c is the y-component of constant acceleration ($\frac{\text{m}}{\text{s}^2}$)
Source	—
RefBy	IM:calOfLandingDist and IM:calOfLandingTime

Detailed derivation of position vector: For a two-dimensional Cartesian coordinate system (**A:twoDMotion** and **A:cartSyst**), we can represent the position vector as $\mathbf{p}(t) =$

$\begin{bmatrix} p_x \\ p_y \end{bmatrix}$, the velocity vector as $\mathbf{v}(t) = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$, and the acceleration vector as $\mathbf{a}(t) = \begin{bmatrix} a_x \\ a_y \end{bmatrix}$. The acceleration is assumed to be constant (**A:constAccel**) and the constant acceleration vector is represented as $\mathbf{a}^c = \begin{bmatrix} a_x^c \\ a_y^c \end{bmatrix}$. The initial velocity (at $t = 0$, from **A:timeStartZero**) is represented by $\mathbf{v}^i = \begin{bmatrix} v_x^i \\ v_y^i \end{bmatrix}$. Since we have a Cartesian coordinate system, **GD:rectPos** can be applied to each coordinate of the position vector to yield the required equation:

$$\mathbf{p}(t) = \begin{bmatrix} p_x^i + v_x^i t + \frac{a_x^c t^2}{2} \\ p_y^i + v_y^i t + \frac{a_y^c t^2}{2} \end{bmatrix}$$

4.2.4 Data Definitions

This section collects and defines all the data needed to build the instance models.

Refname	DD:vecMag
Label	Speed
Symbol	v
Units	$\frac{\text{m}}{\text{s}}$
Equation	$v = \ \mathbf{v}(t)\ $
Description	v is the speed ($\frac{\text{m}}{\text{s}}$) $\mathbf{v}(t)$ is the velocity ($\frac{\text{m}}{\text{s}}$)
Notes	For a given velocity vector $\mathbf{v}(t)$, the magnitude of the vector ($\ \mathbf{v}(t)\ $) is the scalar called speed.
Source	—
RefBy	DD:speedIY and DD:speedIX

Refname	DD:speedIX
Label	x -component of initial velocity
Symbol	v_x^i
Units	$\frac{\text{m}}{\text{s}}$
Equation	$v_x^i = v^i \cos(\theta)$
Description	v_x^i is the x -component of initial velocity ($\frac{\text{m}}{\text{s}}$) v^i is the initial speed ($\frac{\text{m}}{\text{s}}$) θ is the launch angle (rad)
Notes	v^i is from DD:vecMag. θ is shown in Fig:Launch.
Source	—
RefBy	IM:calOfLandingDist

Refname	DD:speedIY
Label	y -component of initial velocity
Symbol	v_y^i
Units	$\frac{\text{m}}{\text{s}}$
Equation	$v_y^i = v^i \sin(\theta)$
Description	v_y^i is the y -component of initial velocity ($\frac{\text{m}}{\text{s}}$) v^i is the initial speed ($\frac{\text{m}}{\text{s}}$) θ is the launch angle (rad)
Notes	v^i is from DD:vecMag. θ is shown in Fig:Launch.
Source	—
RefBy	IM:calOfLandingTime

4.2.5 Instance Models

This section transforms the problem defined in the [problem description](#) into one which is expressed in mathematical terms. It uses concrete symbols defined in the [data definitions](#) to replace the abstract symbols in the models identified in [theoretical models](#) and [general definitions](#).

Refname	IM:calOfLandingTime		
Label	Calculation of landing time		
Input	$v_{\text{launch}}, \theta$		
Output	t_{flight}		
Input Constraints	$v_{\text{launch}} > 0$ $0 < \theta < \frac{\pi}{2}$		
Output Constraints	$t_{\text{flight}} > 0$		
Equation	$t_{\text{flight}} = \frac{2 v_{\text{launch}} \sin(\theta)}{g}$		
Description	t_{flight} is the flight duration (s) v_{launch} is the launch speed ($\frac{\text{m}}{\text{s}}$) θ is the launch angle (rad) g is the magnitude of gravitational acceleration ($\frac{\text{m}}{\text{s}^2}$)		
Notes	The constraint $0 < \theta < \frac{\pi}{2}$ is from A:posXDirection and A:yAxisGravity , and is shown in Fig:Launch . g is defined in A:gravAccelValue . The constraint $t_{\text{flight}} > 0$ is from A:timeStartZero .		
Source	–		
RefBy	IM:calOfLandingDist , FR:Output-Values , and FR:Calculate-Values		

Detailed derivation of flight duration: We know that $p_y^i = 0$ ([A:launchOrigin](#)) and $a_y^c = -g$ ([A:accelYGravity](#)). Substituting these values into the y-direction of [GD:posVec](#) gives us:

$$p_y = v_y^i t - \frac{gt^2}{2}$$

To find the time that the projectile lands, we want to find the t value (t_{flight}) where $p_y = 0$ (since the target is on the x -axis from **A:targetXAxis**). From the equation above we get:

$$v_y^i t_{\text{flight}} - \frac{gt_{\text{flight}}^2}{2} = 0$$

Dividing by t_{flight} (with the constraint $t_{\text{flight}} > 0$) gives us:

$$v_y^i - \frac{gt_{\text{flight}}}{2} = 0$$

Solving for t_{flight} gives us:

$$t_{\text{flight}} = \frac{2v_y^i}{g}$$

From **DD:speedIY** (with $v^i = v_{\text{launch}}$) we can replace v_y^i :

$$t_{\text{flight}} = \frac{2v_{\text{launch}} \sin(\theta)}{g}$$

Refname	IM:calOfLandingDist		
Label	Calculation of landing position		
Input	$v_{\text{launch}}, \theta$		
Output	p_{land}		
Input Constraints	$v_{\text{launch}} > 0$ $0 < \theta < \frac{\pi}{2}$		
Output Constraints	$p_{\text{land}} > 0$		
Equation	$p_{\text{land}} = \frac{2 v_{\text{launch}}^2 \sin(\theta) \cos(\theta)}{g}$		
Description	p_{land} is the landing position (m) v_{launch} is the launch speed ($\frac{\text{m}}{\text{s}}$) θ is the launch angle (rad) g is the magnitude of gravitational acceleration ($\frac{\text{m}}{\text{s}^2}$)		
Notes	The constraint $0 < \theta < \frac{\pi}{2}$ is from A:posXDirection and A:yAxisGravity, and is shown in Fig:Launch. g is defined in A:gravAccelValue. The constraint $p_{\text{land}} > 0$ is from A:posXDirection.		
Source	—		
RefBy	IM:offsetIM and FR:Calculate-Values		

Detailed derivation of landing position: We know that $p_x^i = 0$ ([A:launchOrigin](#)) and $a_x^c = 0$ ([A:accelXZero](#)). Substituting these values into the x-direction of [GD:posVec](#) gives us:

$$p_x = v_x^i t$$

To find the landing position, we want to find the p_x value (p_{land}) at flight duration (from **IM:calOfLandingTime**):

$$p_{\text{land}} = \frac{v_x^i \cdot 2 v_{\text{launch}} \sin(\theta)}{g}$$

From **DD:speedIX** (with $v^i = v_{\text{launch}}$) we can replace v_x^i :

$$p_{\text{land}} = \frac{v_{\text{launch}} \cos(\theta) \cdot 2 v_{\text{launch}} \sin(\theta)}{g}$$

Rearranging this gives us the required equation:

$$p_{\text{land}} = \frac{2 v_{\text{launch}}^2 \sin(\theta) \cos(\theta)}{g}$$

Refname	IM:offsetIM		
Label	Offset		
Input	$p_{\text{land}}, p_{\text{target}}$		
Output	d_{offset}		
Input Constraints	$p_{\text{land}} > 0$ $p_{\text{target}} > 0$		
Output Constraints			
Equation	$d_{\text{offset}} = p_{\text{land}} - p_{\text{target}}$		
Description	d_{offset} is the distance between the target position and the landing position (m) p_{land} is the landing position (m) p_{target} is the target position (m)		
Notes	p_{land} is from IM:calOfLandingDist . The constraints $p_{\text{land}} > 0$ and $p_{\text{target}} > 0$ are from A:posXDirection .		
Source	—		
RefBy	FR:Output-Values and FR:Calculate-Values		

4.2.6 Data Constraints

The **Data Constraints Table** shows the data constraints on the input variables. The column for physical constraints gives the physical limitations on the range of values that can be taken by the variable. The uncertainty column provides an estimate of the confidence with which the physical quantities can be measured. This information would be part of the input if one were performing an uncertainty quantification exercise. The constraints are conservative to

give the user of the model the flexibility to experiment with unusual situations. The column of typical values is intended to provide a feel for a common scenario.

Table 4: Input Data Constraints

Var	Physical Constraints	Typical Value	Uncert.
p_{target}	$p_{\text{target}} > 0$	1000 m	10%
v_{launch}	$v_{\text{launch}} > 0$	100 $\frac{\text{m}}{\text{s}}$	10%
θ	$0 < \theta < \frac{\pi}{2}$	$\frac{\pi}{4}$ rad	10%

4.2.7 Properties of a Correct Solution

The **Data Constraints Table** shows the data constraints on the output variables. The column for physical constraints gives the physical limitations on the range of values that can be taken by the variable.

Table 5: Output Data Constraints

Var	Physical Constraints
p_{land}	$p_{\text{land}} > 0$
d_{offset}	$d_{\text{offset}} > -p_{\text{target}}$
t_{flight}	$t_{\text{flight}} > 0$

5 Requirements

This section provides the functional requirements, the tasks and behaviours that the software is expected to complete, and the non-functional requirements, the qualities that the software is expected to exhibit.

5.1 Functional Requirements

This section provides the functional requirements, the tasks and behaviours that the software is expected to complete.

Input-Values: Input the values from **Tab:ReqInputs**.

Verify-Input-Values: Check the entered input values to ensure that they do not exceed the **data constraints**. If any of the input values are out of bounds, an error message is displayed and the calculations stop.

Calculate-Values: Calculate the following values: t_{flight} (from **IM:calOfLandingTime**), p_{land} (from **IM:calOfLandingDist**), and d_{offset} (from **IM:offsetIM**).

Output-Values: Output t_{flight} (from **IM:calOfLandingTime**) and d_{offset} (from **IM:offsetIM**).

Table 6: Required Inputs

Symbol	Description	Units
p_{target}	Target position	m
v_{launch}	Launch speed	$\frac{\text{m}}{\text{s}}$
θ	Launch angle	rad

5.2 Non-Functional Requirements

This section provides the non-functional requirements, the qualities that the software is expected to exhibit.

Correctness: The outputs of the code have the **properties of a correct solution**.

Verifiability: The code is tested with complete verification and validation plan.

Understandability: The code is modularized with complete module guide and module interface specification.

Reusability: The code is modularized.

Maintainability: If a likely change is made to the finished software, it will take at most 10% of the original development time, assuming the same development resources are available.

Portability: The code shall be portable to multiple environments, particularly Windows, Mac OSX, and Linux.

6 Traceability Matrices and Graphs

The purpose of the traceability matrices is to provide easy references on what has to be additionally modified if a certain component is changed. Every time a component is changed, the items in the column of that component that are marked with an “X” should be modified as well. **Tab:TraceMatAvsA** shows the dependencies of the assumptions on each other. **Tab:TraceMatAvsAll** shows the dependencies of the data definitions, theoretical models, general definitions, instance models, requirements, likely changes, and unlikely changes on the assumptions. **Tab:TraceMatRefvsRef** shows the dependencies of the data definitions, theoretical models, general definitions, and instance models on each other. **Tab:TraceMatAllvsR** shows the dependencies of the requirements and goal statements on the data definitions, theoretical models, general definitions, and instance models.

	A:twoDMotion	A:cartSyst	A:yAxisGravity	A:launchOrigin	A:targetX
A:twoDMotion					
A:cartSyst					
A:yAxisGravity					
A:launchOrigin					
A:targetXAxis					
A:posXDirection					
A:constAccel					
A:accelXZero					
A:accelYGravity			X		
A:neglectDrag					
A:pointMass					
A:freeFlight					
A:neglectCurv					
A:timeStartZero					
A:gravAccelValue					

	A:twoDMotion	A:cartSyst	A:yAxisGravity	A:launchOrigin	A:targetX
DD:vecMag					
DD:speedIX					
DD:speedIY					
TM:acceleration					
TM:velocity					
GD:rectVel					
GD:rectPos					
GD:velVec	X	X			
GD:posVec	X	X			
IM:calOfLandingTime			X	X	X
IM:calOfLandingDist			X	X	

	A:twoDMotion	A:cartSyst	A:yAxisGravity	A:launchOrigin	A:ta
IM:offsetIM					
NFR:Correctness					
NFR:Verifiability					
NFR:Understandability					
NFR:Reusability					
NFR:Maintainability					
NFR:Portability					
FR:Input-Values					
FR:Verify-Input-Values					
FR:Calculate-Values					
FR:Output-Values					

Table 9: Traceability Matrix 5

	DD:vecMag	DD:speedIX	DD:speedIY	TM:acceleration	TM:veloc
DD:vecMag					
DD:speedIX	X				
DD:speedIY	X				
TM:acceleration					
TM:velocity					
GD:rectVel				X	
GD:rectPos					X
GD:velVec					
GD:posVec					
IM:calOfLandingTime			X		
IM:calOfLandingDist		X			
IM:offsetIM					

	DD:vecMag	DD:speedIX	DD:speedIY	TM:acceleration	TM:velocity
GS:targetHit					
NFR:Correctness					
NFR:Verifiability					
NFR:Understandability					
NFR:Reusability					
NFR:Maintainability					
NFR:Portability					
FR:Input-Values					
FR:Verify-Input-Values					
FR:Calculate-Values					
FR:Output-Values					

The purpose of the traceability graphs is also to provide easy references on what has to be additionally modified if a certain component is changed. The arrows in the graphs represent dependencies. The component at the tail of an arrow is depended on by the component at the head of that arrow. Therefore, if a component is changed, the components that it points to should also be changed. [Fig:TraceGraphAvsA](#) shows the dependencies of assumptions on each other. [Fig:TraceGraphAvsAll](#) shows the dependencies of data definitions, theoretical models, general definitions, instance models, requirements, likely changes, and unlikely changes on the assumptions. [Fig:TraceGraphRefvsRef](#) shows the dependencies of data definitions, theoretical models, general definitions, and instance models on each other. [Fig:TraceGraphAllvsR](#) shows the dependencies of requirements and goal statements on the data definitions, theoretical models, general definitions, and instance models. [Fig:TraceGraphAllvsAll](#) shows the dependencies of dependencies of assumptions, models, definitions, requirements, goals, and changes with each other.

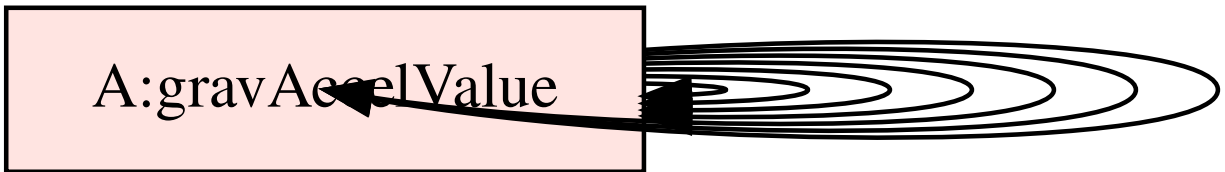


Figure 3: TraceGraphAvsA

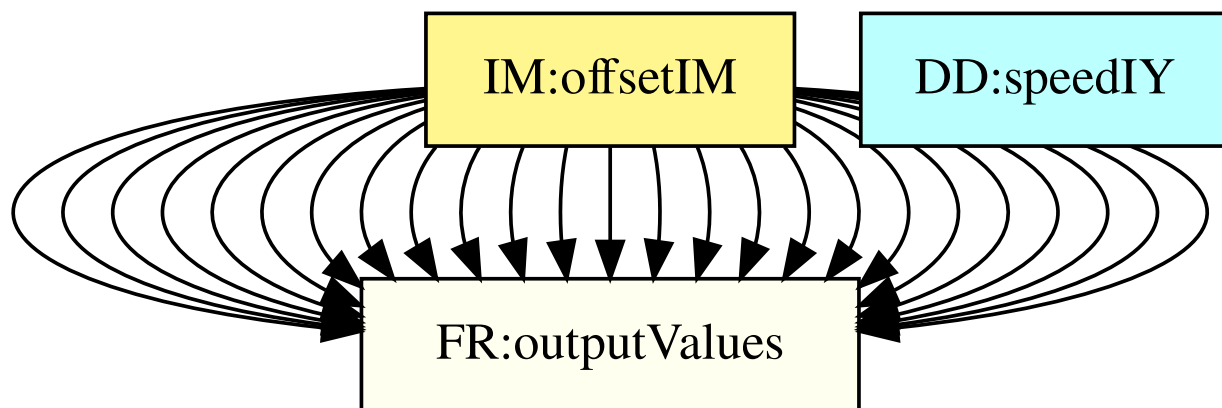


Figure 4: TraceGraphAvsAll

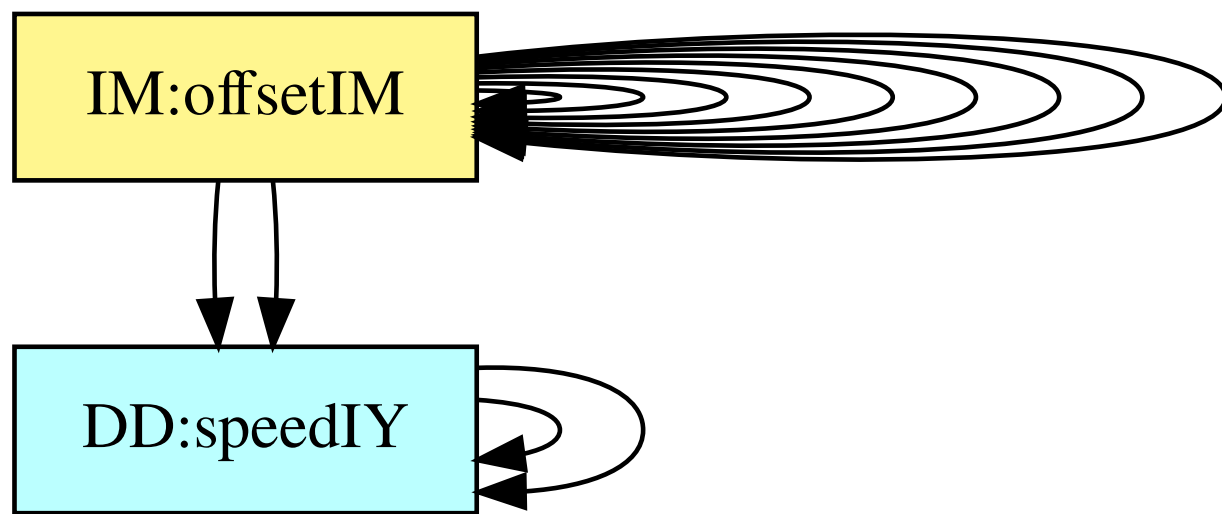


Figure 5: TraceGraphRefvsRef

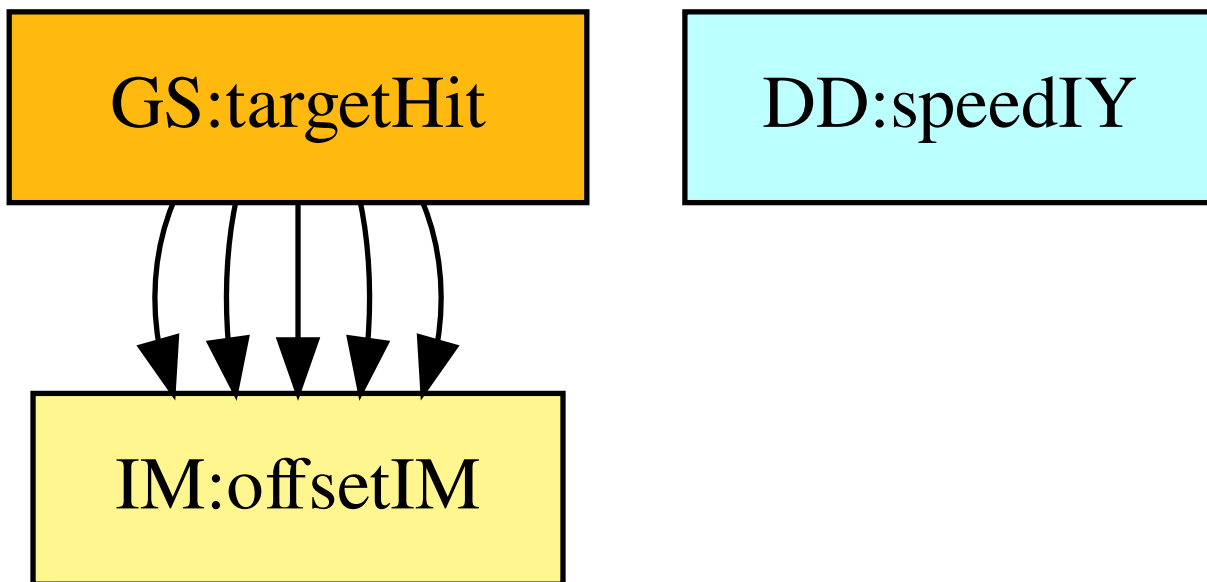


Figure 6: TraceGraphAllvsR

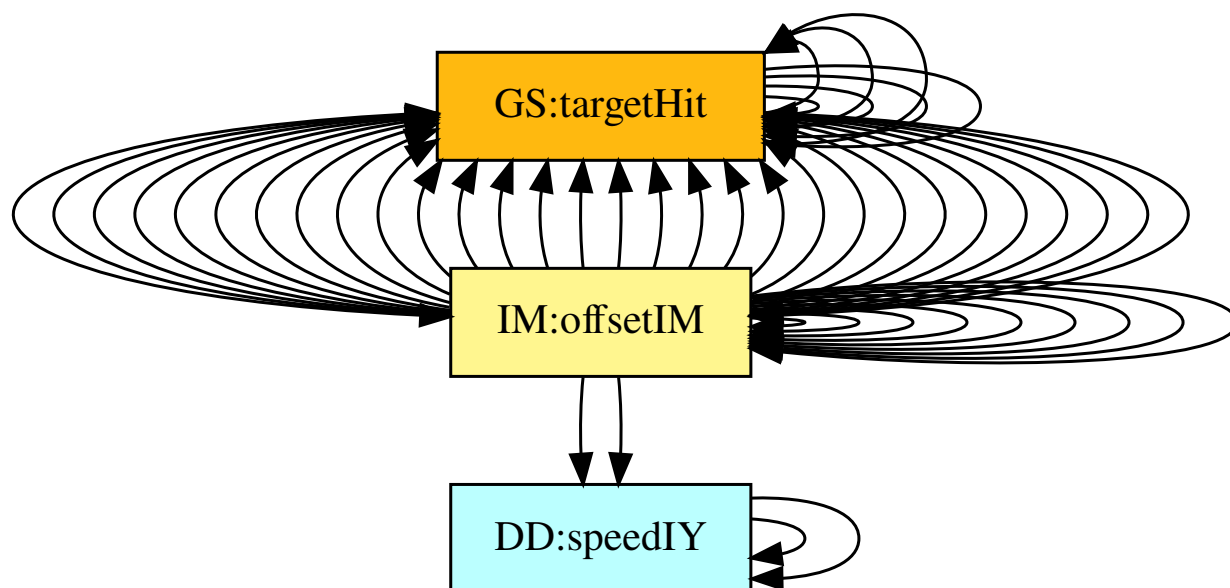


Figure 7: TraceGraphAllvsAll

For convenience, the following graphs can be found at the links below:

- [TraceGraphAvsA](#)

- [TraceGraphAvsAll](#)
- [TraceGraphRefvsRef](#)
- [TraceGraphAllvsR](#)
- [TraceGraphAllvsAll](#)

7 Values of Auxiliary Constants

This section contains the standard values that are used for calculations in Projectile.

Table 11: Auxiliary Constants

Symbol	Description	Value	Unit
g	magnitude of gravitational acceleration	9.8	$\frac{\text{m}}{\text{s}^2}$
ε	hit tolerance	2.0%	—
π	ratio of circumference to diameter for any circle	3.14159265	—

8 References

- [1] Wikipedia Contributors. *Acceleration*. <https://en.wikipedia.org/wiki/Acceleration>. June 2019.
- [2] Wikipedia Contributors. *Cartesian coordinate system*. https://en.wikipedia.org/wiki/Cartesian_coordinate_system. June 2019.
- [3] Wikipedia Contributors. *Velocity*. <https://en.wikipedia.org/wiki/Velocity>. June 2019.
- [4] R. C. Hibbeler. *Engineering Mechanics: Dynamics*. Pearson Prentice Hall, 2004.
- [5] Nirmitha Koothoor. “A Document Driven Approach to Certifying Scientific Computing Software”. MA thesis. Hamilton, ON, Canada: McMaster University, 2013.
- [6] David L. Parnas and P. C. Clements. “A rational design process: How and why to fake it”. In: *IEEE Transactions on Software Engineering* 12.2 (Feb. 1986), pp. 251–257.
- [7] W. Spencer Smith and Nirmitha Koothoor. “A Document-Driven Method for Certifying Scientific Computing Software for Use in Nuclear Safety Analysis”. In: *Nuclear Engineering and Technology* 48.2 (Apr. 2016), pp. 404–418.

- [8] W. Spencer Smith and Lei Lai. “A new requirements template for scientific computing”. In: *Proceedings of the First International Workshop on Situational Requirements Engineering Processes - Methods, Techniques and Tools to Support Situation-Specific Requirements Engineering Processes, SREP’05*. Ed. by PJ Agerfalk, N. Kraiem, and J. Ralyte. In conjunction with 13th IEEE International Requirements Engineering Conference, Paris, France, 2005, pp. 107–121.
- [9] W. Spencer Smith, Lei Lai, and Ridha Khedri. “Requirements Analysis for Engineering Computation: A Systematic Approach for Improving Software Reliability”. In: *Reliable Computing, Special Issue on Reliable Engineering Computation* 13.1 (Feb. 2007), pp. 83–107.